

Session 2: Cryptographic operations and security goals

Slides prepared by UMBC Protocol Analysis Lab

Last edited May 28, 2026

Symmetric encryption

Symmetric encryption

Let m be a message, k be a *secret* key, and c be ciphertext.

Symmetric encryption

Let m be a message, k be a *secret* key, and c be ciphertext.

Definition (Encryption)

A function e is called an *encryption function* if:

$$e(m, k) = c$$

Symmetric encryption

Let m be a message, k be a *secret* key, and c be ciphertext.

Definition (Encryption)

A function e is called an *encryption function* if:

$$e(m, k) = c$$

Definition (Decryption)

A function d is called a *decryption function* if:

$$d(c, k) = m$$

Symmetric encryption

Let m be a message, k be a *secret* key, and c be ciphertext.

Definition (Encryption)

A function e is called an *encryption function* if:

$$e(m, k) = c$$

Definition (Decryption)

A function d is called a *decryption function* if:

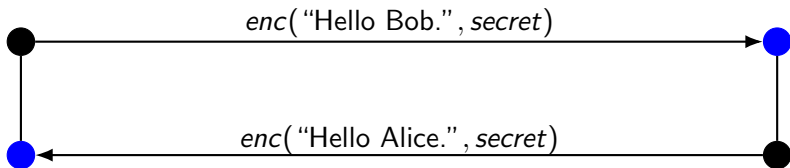
$$d(c, k) = m$$

Common symmetric key algorithms include 3DES, AES, and ChaCha20.

Symmetric Encryption Diagram

Alice

Bob



= Reception



= Transmission

Asymmetric encryption

Asymmetric encryption

Let m be a message, $pub_k, priv_k$ be a public-private key pair, and c be ciphertext.

Asymmetric encryption

Let m be a message, $pub_k, priv_k$ be a public-private key pair, and c be ciphertext.

Definition (Encryption)

A function f is *encrypting* if:

$$f(m, pub_k) = c$$

Asymmetric encryption

Let m be a message, $pub_k, priv_k$ be a public-private key pair, and c be ciphertext.

Definition (Encryption)

A function f is *encrypting* if:

$$f(m, pub_k) = c$$

Definition (Decryption)

A function f is *decrypting* if:

$$f(c, priv_k) = m$$

Asymmetric encryption

Let m be a message, $pub_k, priv_k$ be a public-private key pair, and c be ciphertext.

Definition (Encryption)

A function f is *encrypting* if:

$$f(m, pub_k) = c$$

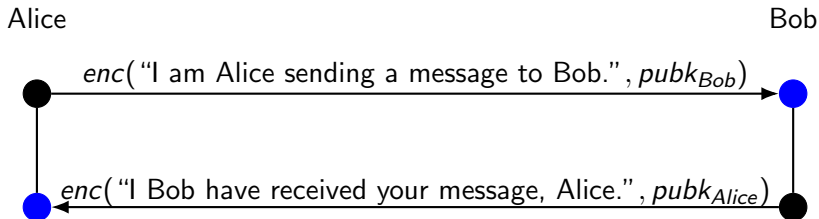
Definition (Decryption)

A function f is *decrypting* if:

$$f(c, priv_k) = m$$

Common asymmetric key algorithms include RSA, ElGamal, and Elliptic Curve.

Asymmetric Encryption Diagram



● = Reception ● = Transmission

Signing

Signing

Let m be a message, $pub_k, priv_k$ be a public-private key pair, and c be ciphertext.

Signing

Let m be a message, $pub_k, priv_k$ be a public-private key pair, and c be ciphertext.

Definition (Signing)

A function f produces a *signature* if:

$$f(m, priv_k) = c$$

Signing

Let m be a message, $pub_k, priv_k$ be a public-private key pair, and c be ciphertext.

Definition (Signing)

A function f produces a *signature* if:

$$f(m, priv_k) = c$$

Definition (Verifying)

A function f can be used to *verify* if:

$$f(c, pub_k) = m$$

Signing

Let m be a message, $pub_k, priv_k$ be a public-private key pair, and c be ciphertext.

Definition (Signing)

A function f produces a *signature* if:

$$f(m, priv_k) = c$$

Definition (Verifying)

A function f can be used to *verify* if:

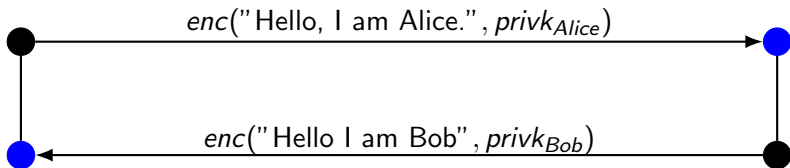
$$f(c, pub_k) = m$$

More or less the same as the asymmetric algorithms.

Signing Diagram

Alice

Bob



= Reception



= Transmission

Hashing

Hashing

Let m be a message, md be a message digest.

Hashing

Let m be a message, md be a message digest.

Definition (Hashing)

A function h is called an *hash function* if:

1. $h(m) = md$
2. It is impractical to determine m given md .
3. Collision Resistance: It is infeasible to find m_1, m_2 such that $h(m_1) = h(m_2)$.

Hashing

Let m be a message, md be a message digest.

Definition (Hashing)

A function h is called an *hash function* if:

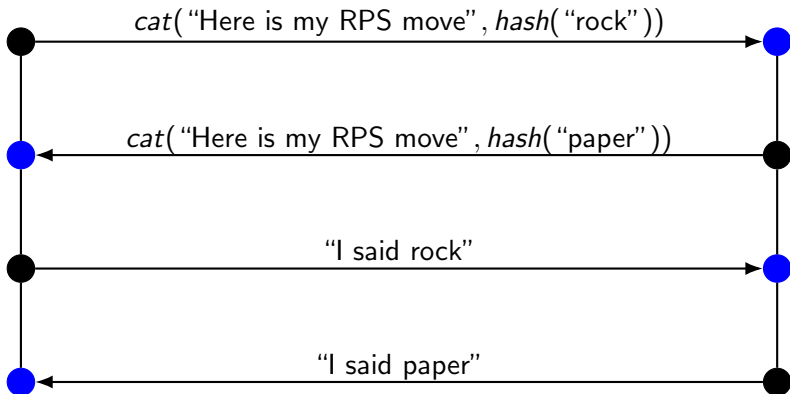
1. $h(m) = md$
2. It is impractical to determine m given md .
3. Collision Resistance: It is infeasible to find m_1, m_2 such that $h(m_1) = h(m_2)$.

Common hash functions include the MD family, SHA family, and Skein.

Hashing Diagram

Alice

Bob



= Reception



= Transmission

Security Goals

How do we know a protocol is secure?

What is a security goal?

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.
Security goals formalize our notion of “secure”.

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.

Security goals formalize our notion of “secure”.

Not every protocol has the same requirement for security.

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.

Security goals formalize our notion of “secure”.

Not every protocol has the same requirement for security.

Air-gapped case:

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.

Security goals formalize our notion of “secure”.

Not every protocol has the same requirement for security.

Air-gapped case: Since there is no one else on the network, we can send the information in plaintext.

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.

Security goals formalize our notion of “secure”.

Not every protocol has the same requirement for security.

Air-gapped case: Since there is no one else on the network, we can send the information in plaintext.

Home network:

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.

Security goals formalize our notion of “secure”.

Not every protocol has the same requirement for security.

Air-gapped case: Since there is no one else on the network, we can send the information in plaintext.

Home network: The adversary is not observing every single packet, but may be able to observe snippets.

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.
Security goals formalize our notion of “secure”.

Not every protocol has the same requirement for security.

Air-gapped case: Since there is no one else on the network, we can send the information in plaintext.

Home network: The adversary is not observing every single packet, but may be able to observe snippets.

Coffee shop:

What is a security goal?

A *security goal* is a goal that we assign for a given protocol to evaluate if it is sufficiently secure for application.

Security goals formalize our notion of “secure”.

Not every protocol has the same requirement for security.

Air-gapped case: Since there is no one else on the network, we can send the information in plaintext.

Home network: The adversary is not observing every single packet, but may be able to observe snippets.

Coffee shop: A lot of people on the network, so we need a very secure protocol.

Common security goals

Common security goals

Confidentiality: In a two-party communication protocol, confidentiality holds if no third party is able to access the information sent.

Common security goals

Confidentiality: In a two-party communication protocol, confidentiality holds if no third party is able to access the information sent.

Agreement: In a two-party communication protocol, agreement holds if both parties agree on some set of data values in the protocol. These values are then used to derive the key.

Common security goals

Confidentiality: In a two-party communication protocol, confidentiality holds if no third party is able to access the information sent.

Agreement: In a two-party communication protocol, agreement holds if both parties agree on some set of data values in the protocol. These values are then used to derive the key.

What security goals might we need for the air-gapped, home, and coffee shop networks?

Other common security goals

1. Aliveness

Other common security goals

1. Aliveness
2. Weak Agreement

Other common security goals

1. Aliveness
2. Weak Agreement
3. Non-injective Agreement

Other common security goals

1. Aliveness
2. Weak Agreement
3. Non-injective Agreement
4. A specific relay party cannot read the message

Other common security goals

1. Aliveness
2. Weak Agreement
3. Non-injective Agreement
4. A specific relay party cannot read the message
5. The message must be timestamped

Other common security goals

1. Aliveness
2. Weak Agreement
3. Non-injective Agreement
4. A specific relay party cannot read the message
5. The message must be timestamped
6. Perfect forward secrecy

Other common security goals

1. Aliveness
2. Weak Agreement
3. Non-injective Agreement
4. A specific relay party cannot read the message
5. The message must be timestamped
6. Perfect forward secrecy
7. Crypto primitives are resistant to a predefined attack

Needham Schroeder

Designing a meeting protocol, and beyond

Strand spaces

A *strand space* is a formalism used to determine the security of a network protocol

- ▶ Fábrega, F. J. T., Herzog, J. C., & Guttman, J. D. (1998, May). [Strand spaces: Why is a security protocol correct?](#)
- ▶ Represent honest communicants and intruders as “strands”
- ▶ Parallel / sequential composability.

Example of a strand

Alice

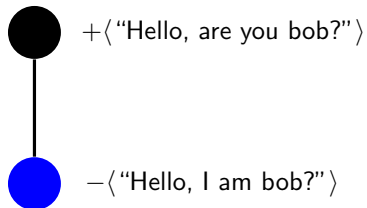
Example of a strand

Alice

 + $\langle \text{"Hello, are you bob?"} \rangle$

Example of a strand

Alice



Strand space

Alice

$+\langle \text{"Hello, are you bob?"} \rangle$ ●

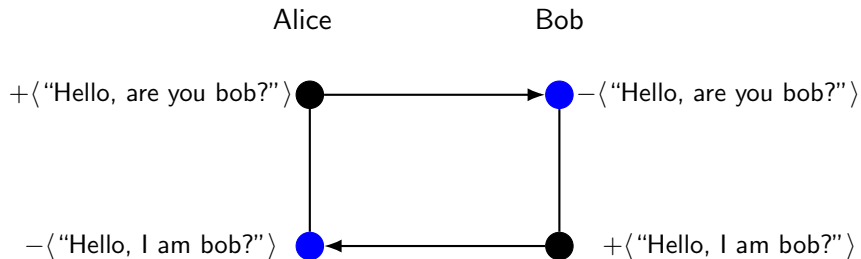
● $-\langle \text{"Hello, I am bob?"} \rangle$

Bob

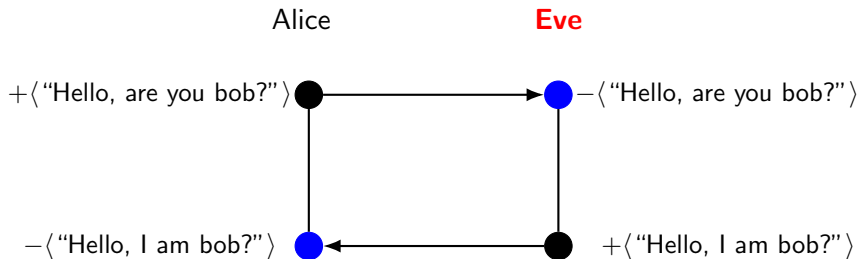
● $-\langle \text{"Hello, are you bob?"} \rangle$

● $+\langle \text{"Hello, I am bob?"} \rangle$

A completed protocol?



Strands can lie



Needham Schroeder

Alice

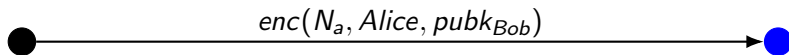
Bob

● = Reception ● = Transmission

Needham Schroeder

Alice

Bob



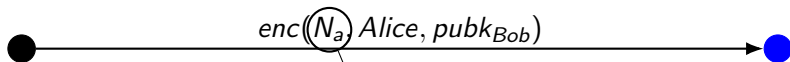
● = Reception

● = Transmission

Needham Schroeder

Alice

Bob



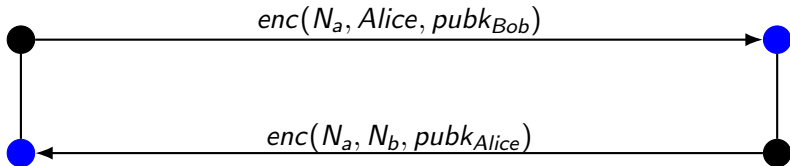
nonce: a *fresh*, cryptographic random number generated by Alice.

● = Reception ● = Transmission

Needham Schroeder

Alice

Bob

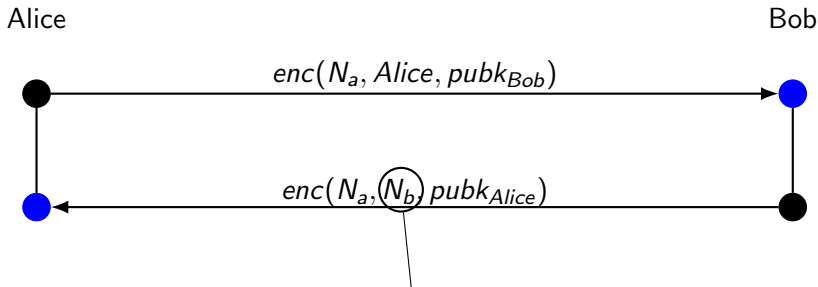


= Reception



= Transmission

Needham Schroeder



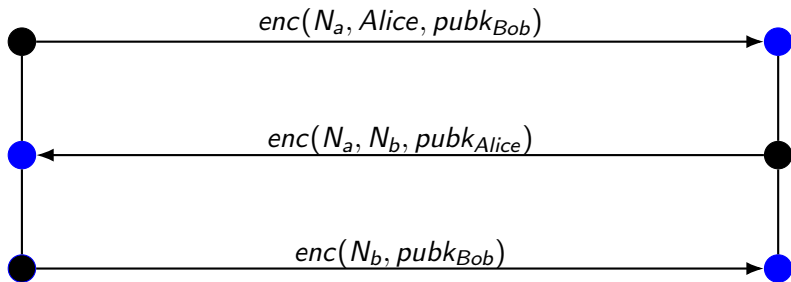
nonce: a *fresh*, cryptographic random number generated by Bob.

● = Reception ● = Transmission

Needham Schroeder

Alice

Bob



= Reception

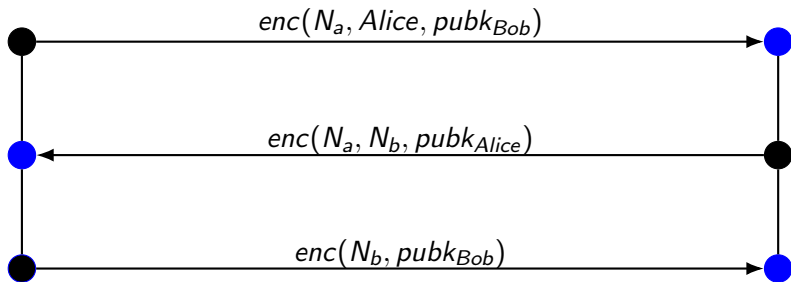


= Transmission

Needham Schroeder

Alice

Bob



Is this protocol secure against the Dolev-Yao Adversary?

● = Reception ● = Transmission

Cryptographic Protocol Shapes Analyzer

- ▶ Tool for analyzing and developing secure protocols.
- ▶ Enumerates all possible executions of a protocol.
- ▶ Assumes a Dolev-Yao adversary.
- ▶ Developed by MITRE, available at:
<https://github.com/mitre/cpsa>

Analyze Needham-Schroeder

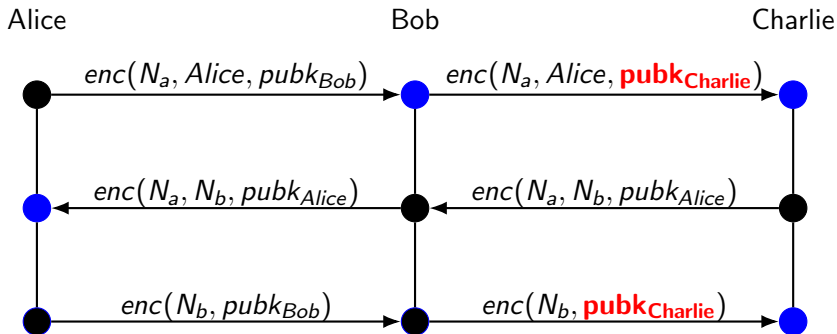
Define protocol *roles* and *skeletons*.

Run CPSA to produce *shapes*.

Analyze *shapes* for expected and unexpected behavior.

Needham Schroeder Attack

Needham Schroeder Attack



Remember: The Dolev-Yao Adversary can be a legitimate party on the network

● = Reception ● = Transmission

Fix your protocol

Fix your protocol

Last time, you created a meeting protocol.

Do you think your meeting protocol withstands the Dolev-Yao adversary?

Fix your protocol

Last time, you created a meeting protocol.

Do you think your meeting protocol withstands the Dolev-Yao adversary?

Model your protocol in CPSA and see if there is an attack!

See if you can fix your protocol so that there is no more attack.

Survey

Fill in details about the survey here!